Deep Reinforcement Learning

TANIGUCHI Taichi

May 30, 2020

Contents

I Preface **2**

II DQN **2**

III TRPO **2**

1 Objectice function **3**
1.1 $L_\pi(\tilde{\pi})$ の定義 3
1.2 conservative policy iteration 4
1.3 TRPO 論文の理論的な主張 5
1.4 制約の導出 6

2 サンプルでの最適化問題の推定 **7**
2.1 期待値での書き換え 7
2.2 サンプル法 8
2.3 Single path 8
2.4 Vine 8

3 実験 **9**
3.1 Simulated Robotic Locomotion 9
3.2 Playing Games from Images 12

4 実装 **12**
4.1 実装の手順 12
4.2 実装 12

5 summary **12**

Part I
Preface

Part II
DQN

Part III
TRPO

Trust Region Policy Optimization, John Schulman, Sergey Levine, Philipp Moritz, Michael I. Jordan, Pieter Abbeelについて、まとめる。

定義 (Preliminaries). 以下がこの TRPO での Notation である。

- ρ_0：初期状態 s_0 の分布
- $(\mathcal{S}, \mathcal{A}, \mathcal{P}, r, \rho_0, \gamma)$：MDP
- $\pi : \mathcal{S} \times \mathcal{A} \to [0, 1]$：確率的方策
- $a_t \sim \pi(a_t|s_t), s_{t+1} \sim \mathcal{P}(s_{t+1}|s_t, a_t)$
- $\eta(\pi) = \mathbb{E}_{s_0, a_0, \ldots}\left[\sum_{t=0}^{\infty} \gamma^t r(s_t)\right]$
- $Q_\pi(s_t, a_t) = \mathbb{E}_{s_{t+1}, a_{t+1}, ..}\left[\sum_{l=0}^{\infty} \gamma^l r(s_t)\right]$
- $V_\pi(s_t) = \mathbb{E}_{a_t, s_{t+1}, \ldots}\left[\sum_{l=0}^{\infty} \gamma^l r(s_t)\right]$
- $A_\pi 8s, a) = Q_\pi(s, a) - V_\pi(s)$

$\eta(\pi)$ は方策 π に従って行動したときの将来においてのる累積報酬の期待値である。Q_π, V_π はそれぞれ state-value function と value function である。

1 Objectice function

定義 (Objective function).

$$\begin{aligned} \max_\theta \quad & L_{\theta_{old}}(\theta) \\ \textit{subject to} \quad & \bar{D}_{KL}^{\rho_{\theta_{old}}}(\theta_{old}, \theta) \leq \delta \end{aligned}$$

ここで、

$$L_{\theta_{old}}(\theta) = \sum_s \rho_{\rho_{\theta_{old}}}(s) \sum_a \pi_\theta(a|s) A_{\theta_{old}}(s,a)$$

$$\bar{D}_{KL}^{\rho_{\theta_{old}}} := \mathbb{E}_{s\sim\rho}[D_{KL}(\pi_{\theta_1})(\cdot|s)||\pi_{\theta_2}(\cdot|s)]$$

以下ではこの目的関数がどのように導出されたかを説明する。

1.1 $L_\pi(\tilde{\pi})$ の定義

まず、方策 $\tilde{\pi}$ について、

$$\eta(\tilde{\pi}) = \eta(\pi) + \mathbb{E}_{s_0,a_0,\ldots\sim\tilde{\pi}}[\sum_{t=0}^{\infty} \gamma^t A_\pi(s_t, a_t)] \tag{1}$$

が成立する。二項目の期待値は行動 a_t が方策 $\tilde{\pi}$ に従って選択された時の期待値を意味する。

次に disconted visitation frequency$\rho_\pi(s)$ を次のように設定する。

$$\rho_\pi(s) = P(s_0 = s) + \gamma P(s_1 = s) + \gamma^2 P(s_2 = s) + \ldots$$

これは初期状態が ρ_0 から生成され、その後方策 π によって、行動が選択された際に、s を訪れる確率を表す。ただし、割引率 γ で割り引いており、正規化もされていないため、確率と呼ぶのはおかしいかもしれない。

これを用いると、方程式1は次のように書き替えられる。

$$\eta(\tilde{\pi}) = \eta(\pi) + \sum_s \rho_{\tilde{\pi}}(s) \sum_a \tilde{\pi}(a|s) A_\pi(s,a) \tag{2}$$

Proof. ********** □

$\sum_a \tilde{\pi}(a|s)A_\pi(s,a)$ が全ての s でに正であれば、$\eta(\tilde{\pi}) >= \eta(\pi)$ であることを示し、つまり期待報酬をより高めるのは方策 $\tilde{\pi}$ であるということがわかる。

方程式2の右辺を最大にさせるような $\tilde{\pi}$ を選択すれば、$\sum_s \rho_{\tilde{\pi}}(s) \sum_a \tilde{\pi}(a|s)A_\pi(s,a)$ だけ方策は改善されるはずである。

しかし、この式を最大化させる $\tilde{\pi}$ を見つけるのは少し難しい。それは、$\tilde{\pi}$ を検証するたびに、$\rho_{\tilde{\pi}}$ での期待値を計算する必要があるからである。

そこで、この $\rho_{\tilde{\pi}}$ での期待値を ρ_π での期待値に置き換えた関数を最適化することにする。つまり、最適化関数 $L_\pi(\tilde{\pi})$ を次のように定義する。

$$L_\pi(\tilde{\pi}) = \eta(\pi) + \sum_s \rho_\pi(s) \sum_a \tilde{\pi}(a|s)A_\pi(s,a)$$

ここからは、$\tilde{\pi}$ や π と書いてきたものを、ある方策クラス

$$\Pi := \{\pi_\theta(a|s) | \theta \in \mathbb{R}^n\}$$

上の方策として表す。

1.2 conservative policy iteration

この関数は $\pi_\theta(a|s)$ が θ で微分可能であれば、η に一次のオーダーで収束することが証明される [1] 。

よって、以下が成立する。

$$L_{\pi_{\theta_0}}(\pi_{\theta_0}) = \eta(\pi_{\theta_0})$$
$$\nabla_\theta L_{\pi_{\theta_0}}(\pi_\theta)|_{\theta=\theta_0} = \nabla_\theta \eta(\pi_\theta)|_{\theta=\theta_0}$$

[1] では、$\pi^{'} \in \arg\max_\pi L_{\pi_{old}}(\pi)$ として、$\pi_{new}(a|s)$ を

$$\pi_{new}(a|s) = (1-\alpha)\pi_{old}(a|s) + \alpha\pi^{'}(a|s)$$

とすると、以下の不等式が成立することを示している。

$$\eta(\pi_{new}) \geq L_{\pi_{old}}(\pi_{new}) - \frac{2\epsilon\gamma}{(1-\gamma)^2}\alpha^2$$

ただし、$\epsilon = max_s |\mathbb{E}_{a\sim\pi'(a|s)}[A_\pi(a,s)]|$

1.3 TRPO 論文の理論的な主張

カテゴリカル分布 p, q に対して、

$$D_{TV}(p||q) = \frac{1}{2}\sum_i |p_i - q_i|$$

と定義する。さらに、$D_{TV}^{max}(\pi, \tilde{\pi})$ を

$$D_{TV}^{max}(\pi, \tilde{\pi}) = \max_s D_{TV}\left(\pi(\cdot|s)||\tilde{\pi}(\cdot|s)\right)$$

と定義する。この時次の不等式が成立する。

定義 (TRPO 論文の理論的主張). $\alpha = D_{TV}^{max}(\pi_{old}, \pi_{new})$ とすると、次の不等式が成立する。

$$\eta(\pi_{new}) \geq L_{\pi_{old}}(\pi_{new}) - \frac{4\epsilon\gamma}{(1-\gamma)^2}\alpha^2$$

ただし、$\epsilon = \max_{s,a} |A_\pi(s,a)|$

また、D_{KL} を KL ダイバージェンスとすると、

$$D_{TV}(p||q)^2 \leq D_{KL}(p||q)$$

が成立する。

$$D_{KL}^{max}(\pi, \tilde{\pi}) = \max_s D_{KL}(\pi(\cdot|s)||\tilde{\pi}(\cdot|s))$$

と定義すると、上の不等式は以下のように書き換えられる。

定義 (TRPO 論文の理論的主張の書き換え).

$$\eta(\tilde{\pi}) \geq L_\pi(\tilde{\pi}) - CD_{KL}^{max}(\pi, \tilde{\pi})$$

ただし、$C = \frac{4\epsilon\gamma}{(1-\gamma)^2}$

i 回目の更新で得た方策を π_i とすると、第 i+1 回目の更新を

$$\pi_{i+1} = \arg\max_\pi [L_{\pi_i}(\pi) - CD_{KL}^{max}(\pi_i, \pi)]$$

で得るアルゴリズムを考える。

$$M_i(\pi) = L_{\pi_i}(\pi) - CD_{KL}^{max}(\pi_i, \pi)$$

とすると、

$$\begin{aligned} \eta \quad (\pi_{i+1}) &\geq M_i(\pi_{i+1}) \\ \eta \quad (\pi) &= M_i(\pi_i) \end{aligned}$$

が成立する。よって、

$$\eta(\pi_{i+1}) - \eta(\pi_i) \geq M_i(\pi_{i+1}) - M(\pi_i)$$

が成立する。これはつまり、このアルゴリズムで得られた方策の数列が期待報酬 η を単調に改善していくことを示している。

1.4 制約の導出

ここから先は π_θ などは全て θ と表すことにする。例えば、$\eta(\theta) := \eta(\pi_\theta)$ である。

先ほどのアルゴリズムでは

$$\arg\max_\theta [L_{\theta_{old}}(\theta) - CD_{KL}^{max}(\theta_{old}, \theta)]$$

を解いて、その解の一つを次の方策として更新していけば、単調改善が可能であった。

この更新はある意味 $L_{\theta_{old}}(\theta)$ という目的関数に対して距離関数 (みたいなもの)D_{KL}^{max} によって θ の正則化を行っていると見れる。

ただし、実問題上では、ペナルティー項である C はかなり小さいものが推奨されている。(と TRPO の論文には書かれている)。そこで、上の最適化問題を次のように書き換える。

$$\begin{aligned} \max_\theta \quad & L_{\theta_{old}}(\theta) \\ subject\ to \quad & D_{KL}^{max}(\theta_{old}, \theta) \leq \delta \end{aligned}$$

ただし、max に制約があるということは、すべての状態 s に対して制約があることと同値である。これでは制約が多くなりすぎるので、期待値にのみ制約を設ける。

$$\bar{D}_{KL}^{\rho}(\theta_1, \theta_2) := \mathbb{E}_{s\sim\rho}[D_{KL}(\pi_{\theta_1}(\cdot|s)||\pi_{\theta_2}(\cdot|s)]$$

よって、最適化問題は次のようになる。

定義 (制約問題への書き換え).

$$\begin{aligned} \max_\theta \quad & L_{\theta_{old}}(\theta) \\ subject\ to \quad & \bar{D}_{KL}^{max}(\theta_{old}, \theta) \leq \delta \end{aligned}$$

後の実験結果で示すことだが、このようにして、全ての s に対して制約を設けるのと、期待値のみに制約を設けるので、期待報酬は同程度までしか上がらないことが実証されている。

2 サンプルでの最適化問題の推定

2.1 期待値での書き換え

上の最適化問題は目的関数を明示的に書くと、

$$\begin{aligned} \max_\theta \quad & \sum_s \rho_{\theta_{old}}(s) \sum_a \pi_\theta(a|s) A_{\theta_{old}}(s,a) \\ subject\ to \quad & \bar{D}_{KL}^{max}(\theta_{old}, \theta) \leq \delta \end{aligned}$$

となる。

これを数値計算によって解くためには、最適化問題を期待値での記号で置き換え、サンプル近似により最適化問題を計算する必要がある。

$\sum_s \rho_{\theta_{old}}(s)[\cdot]$ を $\frac{1}{1-\gamma}\mathbb{E}_{s\sim\rho_{\theta_{old}}}[\cdot]$ で置き換える。すると、目的関数は以下のようになる。

$$\max_\theta \mathbb{E}_{s\sim\rho_{\theta_{old}}}\left[\sum_a \pi_\theta(a|s) A_{\theta_{old}}(s,a)\right]$$

さらに、$\sum_a \pi_\theta(a|s) A_{\theta_{old}}(s,a)$ の部分を $\pi(a|s)$ の期待値に直して、importance sampling を分布 $q(a|s)$ で用いて、制約部分も明示的に書くようにすると、最適化問題は次のように書きなおせる。

定義 (期待値での最適化問題).

$$\max_\theta \mathbb{E}_{s\sim\rho_{\theta_{old}}, a\sim q}\left[\frac{\pi_\theta(a|s)}{q(a|s)} Q_{old}(s,a)\right]$$

$$subject\ to\ \mathbb{E}_{s\sim\rho_{\theta_{old}}}\left[D_{KL}(\pi_{old}(\cdot|s)||\pi_\theta(\cdot|s)\right] \leq \delta$$

ただし、$A_{old}(s,a)$ を $Q_{old}(s,a)$ に置き換えた。(なぜ置き換えていいのか？)

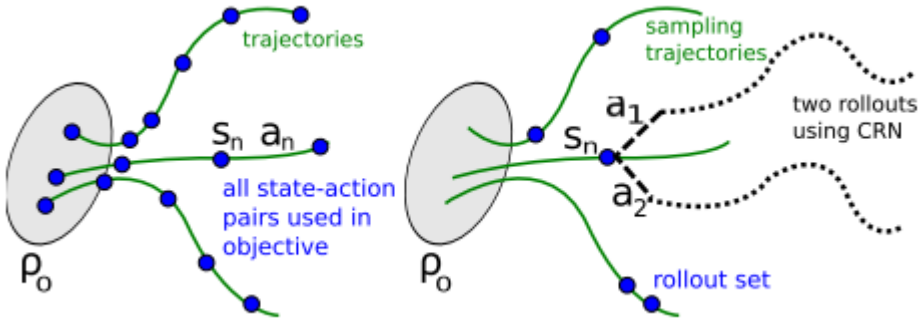

Figure 1: Single path and vine

2.2 サンプル法

上で、最適化問題を期待値を使って書き換えた。後は、これをサンプルによって近似する。このサンプルの取得方法には二つの方法があり、それが single path と vine という方法である。Figure1はそれぞれのサンプル法を表す。左が single path で右が vine である。

2.3 Single path

single path では以下のようにしてサンプルを生成する。

- $s_0 \sim \rho_0$
- $\pi_{\theta_{old}}(a_t|s_t)$ によって a_t を確率的に決定
- $\mathcal{P}(s_{t+1}|s_t, a_t)$ によって次の状態 s_{t+1} を確率的に決定
- 上の二つの処理を繰り返す。

このようにしてできた trajectory $s_0, a_0, s_1, a_1, ..., s_{T-1}, a_{T-1}, s_T$ を用いて状態と行動のペアを作成し、$Q_{\theta_{old}}(s, a)$ を計算し、そのあと、期待値を近似する。

2.4 Vine

vine では以下のようにしてサンプルを生成する。

- $s_0 \sim \rho_0$
- single path の時と同様に trajectory を一つ生成
- trajectory に出てきた状態の中から N 個の状態 $\{s_1, s_2, ...s_N\}$ を選択 (それを集めた集合を rollout と呼ぶ。)

- 一つの $s_n \in rollout$ に対して、K 個の行動 $\{a_{n,1}, a_{n,2}, ..., a_{n,K}\}$ を $q(\cdot|s_n)$ から生成
- $\mathcal{P}$ によって、s_n に対して、K 個の行動 $a_{n,1}, a_{n,2}, ..., a_{n,K}$ をとった時の次の状態を生成。
- 上三つを繰り返す

一つの状態 s_n に対して、行動のサンプルが $\{a_{n,1}, a_{n,2}, ..., a_{n,K}\}$ の K 個できたことになる。

スタートの状態が s_n で、行動が $a_{n,k}$ で始まった short trajectory によって、計算できる Q-value を $\hat{Q}(s_n, a_{n,k}$ とする。

これを用いて Q-value を推定するための式は

$$L_n(\theta) = \frac{\sum_{k=1}^{K} \frac{\pi_\theta(a_{n,k}|s_n)}{\pi_{\theta_{old}}(a_{n,k}|s_n)} \hat{Q}(s_n, a_{n,k})}{\sum_{k=1}^{K} \frac{\pi_\theta(a_{n,k}|s_n)}{\pi_{\theta_{old}}(a_{n,k}|s_n)}}$$

vine は single path よりも目的関数の分散を小さくすることができる。しかし、vine はアドバンテージ関数の推定のたびにシュミレーションを行わなければならない、システムを任意の状態へリセットできる問題でないと適用できない、などの問題点がある。

3 実験

以下のような疑問に対する実験的な答えを得る。

- single path と vine でのパフォーマンスの違いは？
- KL での制約にしたことにより、従来手法とどう違ってくるか？
- TRPO はパフォーマンス、計算時間、サンプルの複雑さの三つの観点で従来の手法と比べる。

3.1 Simulated Robotic Locomotion

Table1の三種類の問題を考える。

Table 1: robotic locomotion model

motion	s-dim	reward
Swimmer	10	$r(x,u) = v_x - 10^{-5}\|\|u\|\|^2$
Hopper	12	$r(x,u)$ に倒れてない間は常に $+1$ のボーナスを追加
Walker	18	$r(x,u)$ に足への衝撃へのペナルティを追加

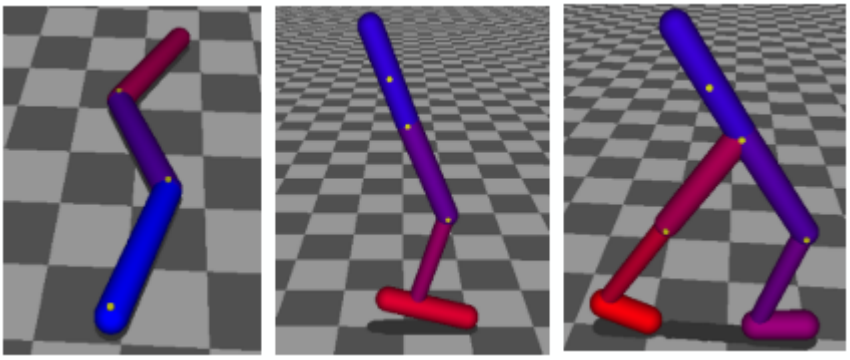

Figure 2: robotic simulation

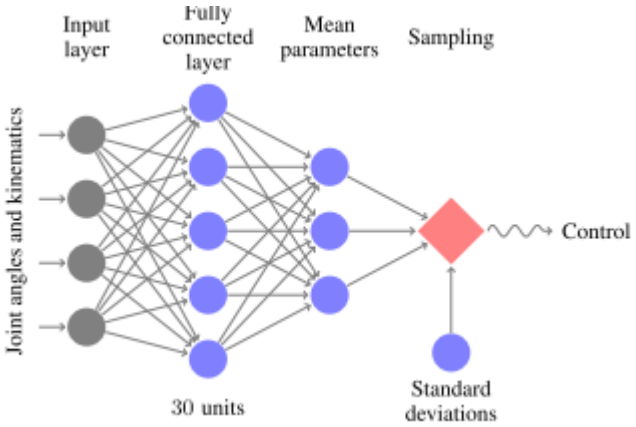

Figure 3: NN for robotic locomotion

Figure2は左から swimming, hopper,Walker のシュミレーションのモデルである。この三つにさらに、典型的な cart-pole の問題を追加。

全ての実験において、$\delta = 0.01$ である。

また、policy は Figure3で示された NN で近似する。

Figure4は報酬関数の推移が手法毎に plot されたものがそれぞれの task で並んでいる。この Figure4から読み取れることをまとめる。

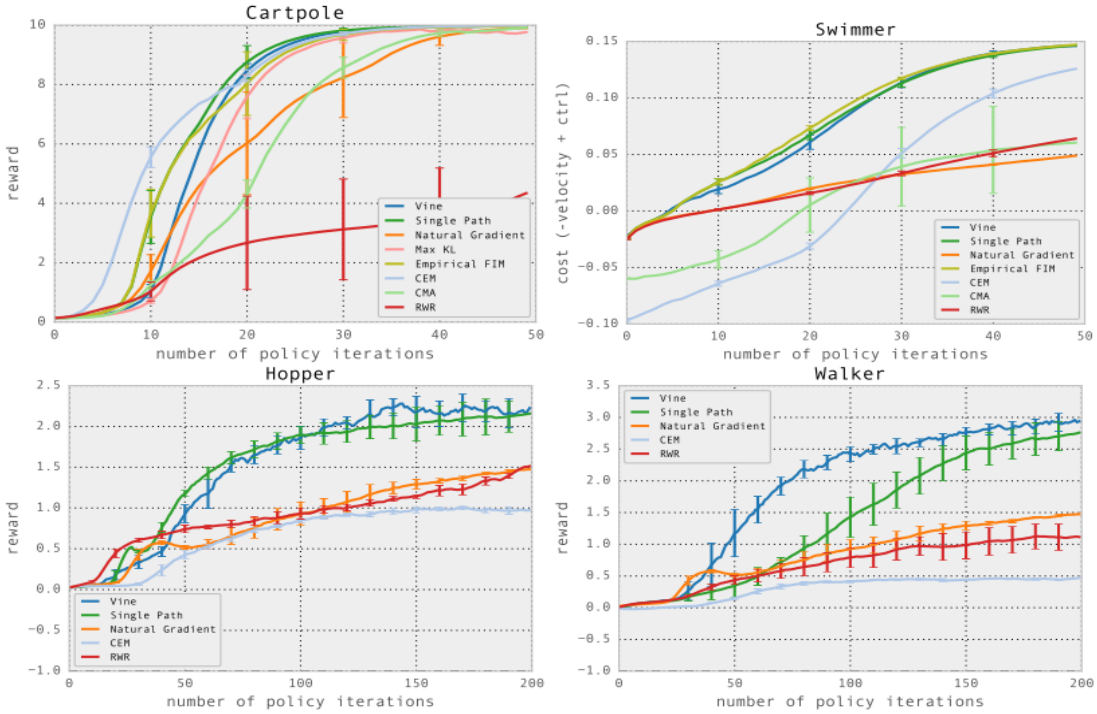

Figure 4: Learning curve

- cort-porl：KL^{max} への制約で実装したのが、Max KL という曲線だが、single path や vine と同程度までしか期待報酬は上がっていないので、KL 制約は平均のみに課しても大丈夫 (なはず)
- natural policy gradient は cart-porl、swimming ではうまくいってるが、walking、hopping ではうまくいっていない。よって、TRPO はよりロバスト性を備えている。(なぜ walking,hopping 問題はロバスト性を確認できる問題になっている？)
- CEM と CMA は cor-pool 問題のみうまくいっている。しかしより複雑な問題である walking,hopping,swwiming では精度は良くない。

	B. Rider	*Breakout*	*Enduro*	*Pong*	*Q*bert*	*Seaquest*	*S. Invaders*
Random	354	1.2	0	−20.4	157	110	179
Human (Mnih et al., 2013)	7456	31.0	368	−3.0	18900	28010	3690
Deep Q Learning (Mnih et al., 2013)	4092	168.0	470	20.0	1952	1705	581
UCC-I (Guo et al., 2014)	5702	380	741	21	20025	2995	692
TRPO - single path	1425.2	10.8	534.6	20.9	1973.5	1908.6	568.4
TRPO - vine	859.5	34.2	430.8	20.9	7732.5	788.4	450.2

Figure 5: Atari geme experiment result

3.2 Playing Games from Images

画像から Atari ゲームをプレイするための方策を学習する実験。状態空間、行動空間の次元が高いことと、報酬が遅延により減少していくのがこの問題の難しい点である。

Figure5は人間、Deep Q learning, UCCC-I と TRPO を比べたものである。他のアルゴリズムが特定の task に特化するように事前情報などをプログラムされているが、TRPO には task 特有の事前情報をプログラムする必要がない。このことを考慮すると、TRPO はそれぞれの task でまずまずの結果を示せている。

4 実装

4.1 実装の手順

- single path もしくは vine を使ってサンプルを生成し、モンテカルロ推定で Q-value を計算
- サンプルを平均することで、目的関数と制約関数を近似
- 共役勾配法と直線探索により、最適化問題を解く。

4.2 実装

5 summary

- KL 制約により、よりロバストな推定を可能にした。
- 事前情報 0 から方策を学習するための初めての手法
- 実験的に事前情報を組み込んだこれまでの手法よりも汎用性が高い

References

[1] Kakade, Sham and Langford, John. Approximately optimal ap- proximate reinforcement learning. In ICML, volume 2, pp. 267-274, 2002.